PROVA FINALE

RETI LOGICHE

2023/24

Prof William Fornaciari

Gruppo da 2 studenti:

- **Ramiro Cicero (Codice Persona: 10710719 - Matricola: 957530)**
- **Margherita Bernasconi (Codice Persona: 10664239 - Matricola: 935382)**

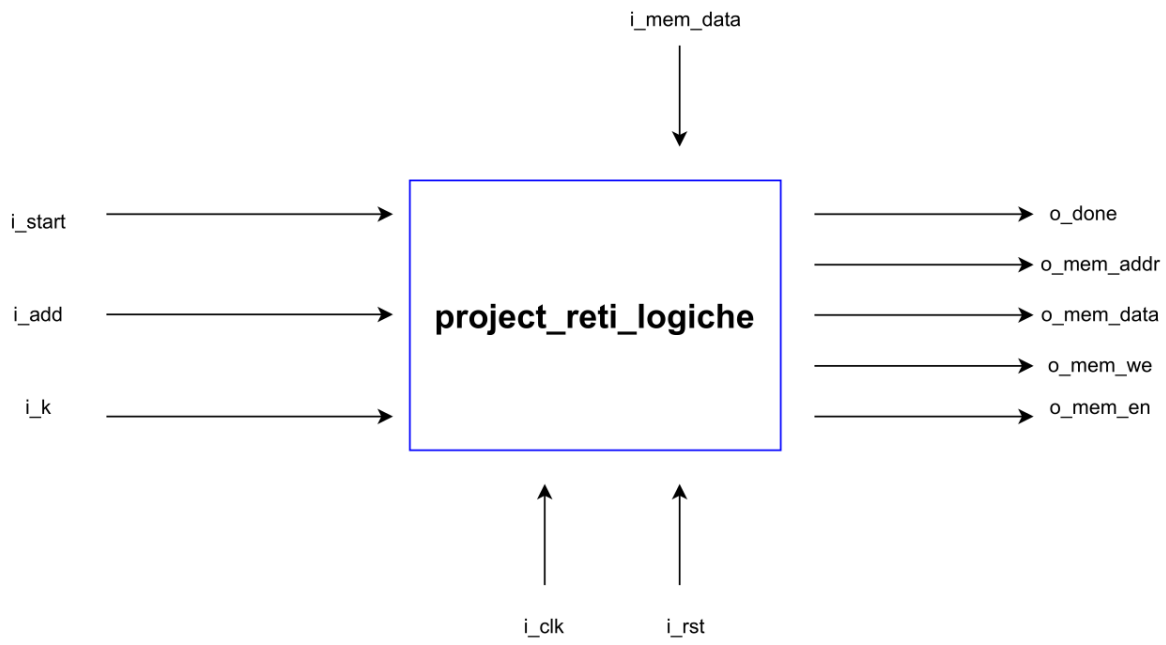
INTRODUZIONE

La "Prova Finale (Progetto di Reti Logiche) 2023/24" prevede la progettazione di un modulo hardware in VHDL, capace di interfacciarsi con una memoria RAM e modificare specifici dati in essa contenuti

DESCRIZIONE MODULO

Il modulo principale, denominato "project_reti_logiche", è descritto come segue:

(Immagine 1)



SPECIFICA

Il modulo deve essere in grado di aggiornare i_k parole nella memoria RAM a partire dall'indirizzo i_add. Tra una parola e l'altra, è presente un byte libero (impostato a 0). L'aggiornamento della memoria inizia quando i_start diventa '1', e al termine dell'operazione, o_done deve essere impostata a '1'. Successivamente, quando i_start ritorna a '0', o_done deve essere riportato a '0'.

Per leggere un dato dalla RAM, il modulo deve impostare o_mem_en a '1' e o_mem_we a '0', e specificare l'indirizzo di lettura in o_mem_addr. Per scrivere nella memoria, sia o_mem_en che o_mem_we devono essere impostati a '1', con o_mem_addr che indica l'indirizzo di scrittura e o_mem_data il dato da memorizzare.

Il modulo aggiornerà la RAM sostituendo i valori zero (esclusi i byte mancanti) con l'ultima parola non zero trovata. I byte mancanti saranno riempiti con un valore di credibilità iniziale di 31, decrementato di 1 ogni volta che si incontra uno zero. Se viene trovato un valore diverso da zero, il valore di credibilità sarà resettato a 31; finché non ci sono valori diversi da zero, il valore di credibilità sarà 0.

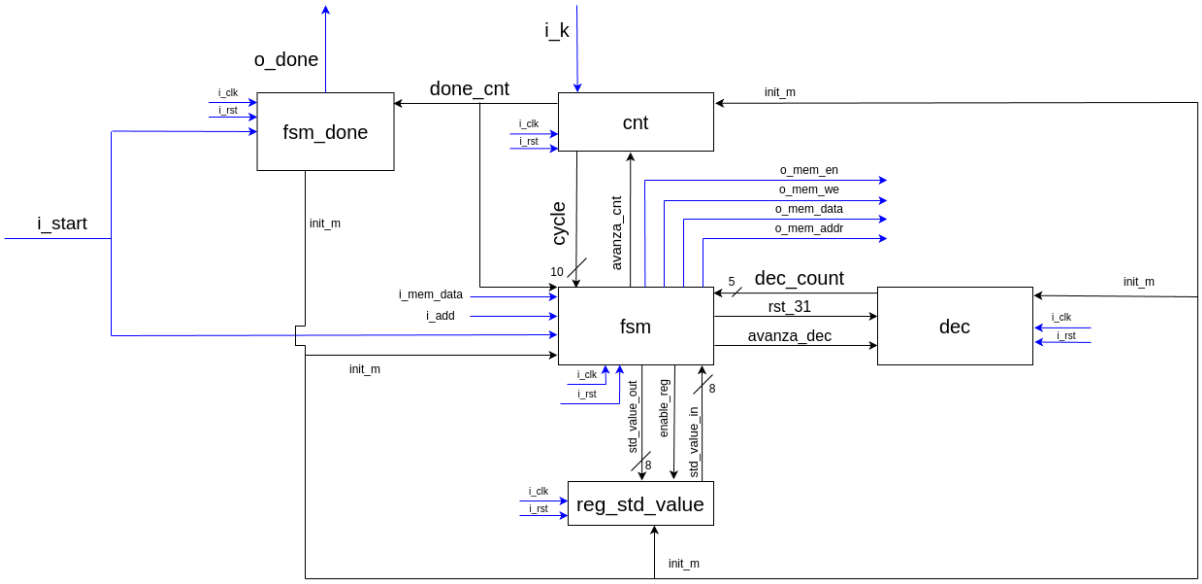
In caso di reset (i_rst, segnale asincrono), tutti i valori saranno riportati allo stato iniziale e il processo riprenderà quando i_start sarà nuovamente impostato a 1.

Il modulo deve operare con un clock di almeno 20 ns.

Tutti questi segnali avvengono in modo sincrono, e devono essere interpretati sul fronte di salita del clock (escluso i_rst).

DESIGN

La struttura generale dei moduli è rappresentata come segue:



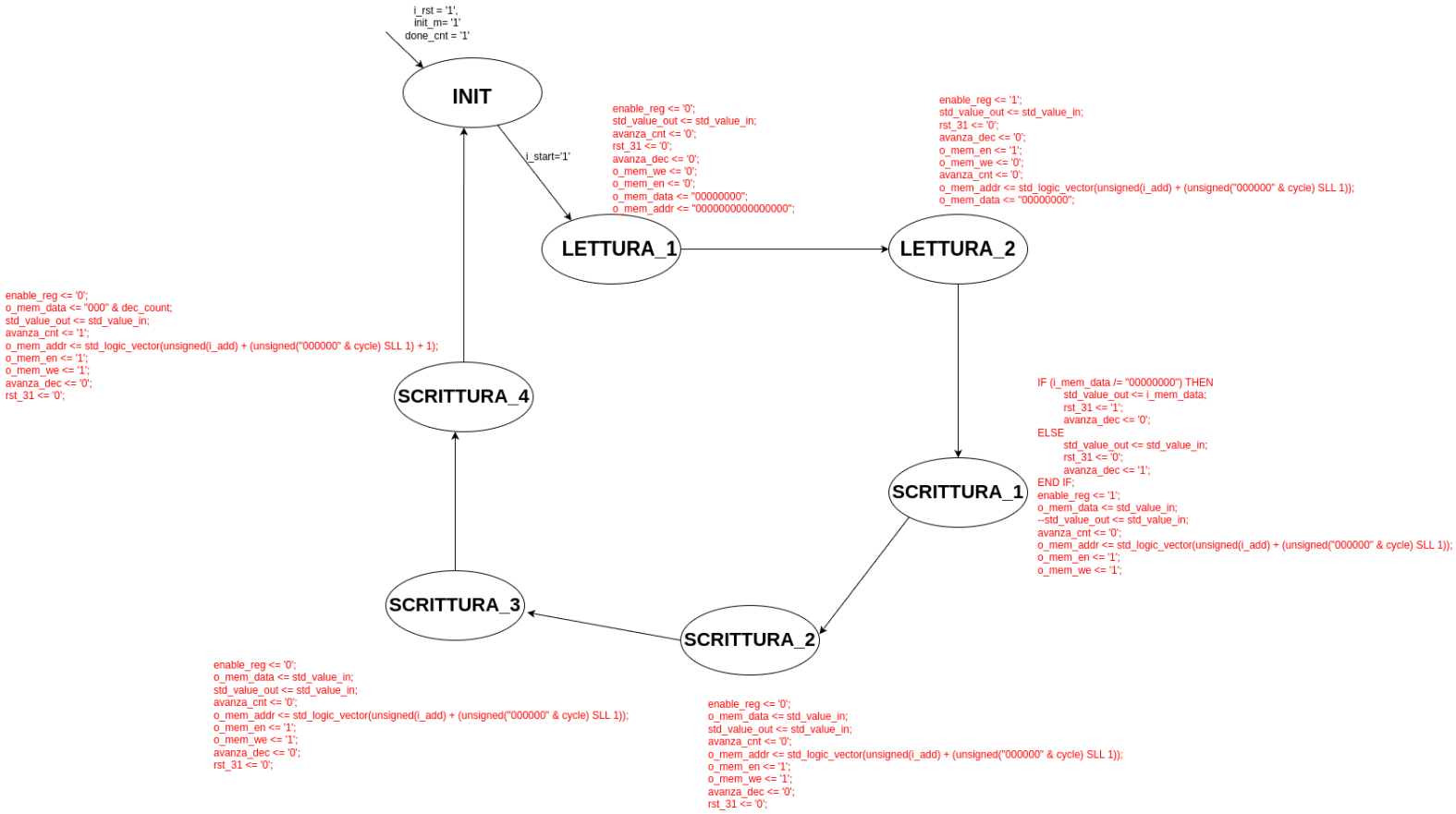
(Immagine 2)

I segnali in blu indicano quelli relativi all'entity principale (Immagine 1).

Quando il segnale i_start viene impostato a '1' il modulo FSM (Finite State Machine) avvia il proprio ciclo, che si sviluppa attraverso i seguenti stati:

- **INIT**: Questo stato funge sia da stato iniziale che da stato di reset. Tutti i segnali vengono azzerati, con std_value_in impostato uguale a std_value_out. Se il segnale i_rst viene portato a '1', lo stato attuale (curr_state) della FSM torna a INIT.
- **LETTURA_1**: Viene attivata la lettura dalla RAM e viene specificato l'indirizzo da cui estrarre il dato.
- **LETTURA_2**: Viene attivato l'aggiornamento di std_value (spiegato in seguito)
- **SCRITTURA_1**: Si attiva la scrittura nella RAM. Se std_value_out è diverso da zero, il valore di credibilità viene resettato a 31, altrimenti viene decrementata la credibilità, e infine viene scritto il dato nel primo byte.
- **SCRITTURA_2**: Viene disattivato l'aggiornamento di std_value.
- **SCRITTURA_3**: Simile a SCRITTURA_2, questo stato viene mantenuto per permettere a o_mem_data di aggiornarsi correttamente.
- **SCRITTURA_4**: Viene indicato il valore di credibilità e l'indirizzo in cui deve essere salvato, mentre il contatore (cnt) viene incrementato.

Il passaggio tra gli stati avviene sul fronte di salita del clock. Invece il modulo transita da INIT a LETTURA_1 solo quando i_start è '1' (fronte di salita del clock).



Moduli Ausiliari

A supporto della FSM, ci sono alcuni moduli ausiliari:

- **reg_std_value**: Registro che riceve in ingresso std_value_out della FSM. Se il segnale enable_reg è '1', il valore di uscita (std_value_in della FSM) viene aggiornato. Questo modulo consente di memorizzare l'ultimo valore diverso da zero, utile per scriverlo in memoria quando si incontrano zeri.
    - i_rst = '1': il segnale in uscita viene azzerato.
- **dec**: Contatore inverso che decrementa il valore di dec_count (credibilità) di 1 quando avanza_dec = '1' e dec_count è diverso da zero. Se il segnale rst_31 è '1', il valore viene resettato a 31.
    - i_rst = '1': il segnale in uscita viene azzerato.
- **cnt**: Contatore che incrementa il segnale di uscita (cycle) di 1 quando avanza_cnt = '1'.
    - i_rst = '1': il segnale in uscita viene azzerato.

Quando cycle raggiunge il valore i_k - 1, il modulo cnt imposta il segnale done_cnt a '1'.

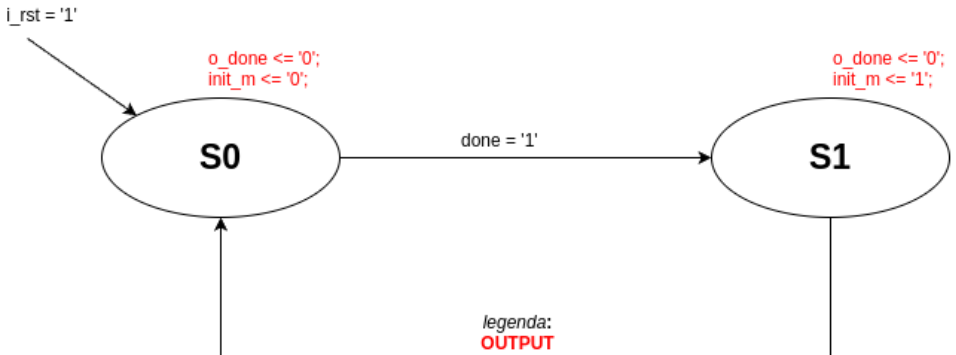
Al termine del ciclo, interviene il modulo *fsm_done*, una macchina a stati finiti con i seguenti stati:

- **S0**: Stato di reset. Rimane in questo stato fino a quando done_cnt = '0'. Se done_cnt = '1', lo stato passa a S1.
  - **OUTPUT**: o_done = '0'.
  - **OUTPUT**: init_m = ‘0’.
- **S1**: Rimane in questo stato fino al prossimo fronte di salita del clock
  - **OUTPUT**: o_done = '0'.
  - **OUTPUT**: init_m = ‘1’.

- **S2**: Rimane in questo stato fino al prossimo fronte di salita del clock ove i_start vale 0
  - **OUTPUT**: o_done = '1'.
  - **OUTPUT**: init_m = ‘0’.
- **S3**: Lo stato ritorna a S0 al successivo fronte di salita del clock.
  - **OUTPUT**: o_done = '0'.
  - **OUTPUT**: init_m = ‘1’.

L'obiettivo di questo modulo è di impostare o_done a '1' al termine dell'operazione, attendere che il segnale i_start torni a '0', e poi riportare o_done a '0'.Inoltre viene generato un segnale init_m che serve a resettare tutti i moduli (cnt, fsm, dec,reg_std_value), in modo tale che possa essere analizzata una successiva sequenza.

Il modulo attende quindi il prossimo segnale di "start". Si noti che tutti i segnali interni agiscono sul fronte di salita del clock. Inoltre, il reset e il clock di ciascun modulo sono collegati rispettivamente a i_rst e i_clk.



Risultati Sperimentali

La sintesi del modulo è stata eseguita con successo, soddisfacendo due aspetti fondamentali del progetto:

- Attraverso la **Tcl Console**, utilizzando il comando *report_utilization*, è stato verificato che non sono stati generati latch indesiderati, potenziali fonti di malfunzionamenti.
- Sempre tramite la **Tcl Console**, con il comando *report_timing*, è stato confermato che il vincolo temporale del clock (20 ns) è stato rispettato. Il ritardo massimo misurato è infatti pari a soli 4,249 ns, al di sotto del limite richiesto.

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	86	0	0	134600	0.06
LUT as Logic	86	0	0	134600	0.06
LUT as Memory	0	0	0	46200	0.00
Slice Registers	31	0	0	269200	0.01
Register as Flip Flop	31	0	0	269200	0.01
Register as Latch	0	0	0	269200	0.00
F7 Muxes	0	0	0	67300	0.00
F8 Muxes	0	0	0	33650	0.00

```
Timing Report

Slack (MET) :             15.751ns  (required time - arrival time)
```

Test Bench

Il modulo è stato sottoposto a cinque diversi test bench:

Test Bench 1

(TB fornito dal docente)

In questo test viene simulato uno scenario "normale", in cui la RAM contiene una sola sequenza lunga i_k. Dopo il reset, il modulo deve modificare la sequenza seguendo le regole specificate in precedenza.

- Dopo il reset, o_done è impostato a '0', come previsto, poiché viene azzerato nel modulo fsm_done.
- I segnali o_mem_we e o_mem_en sono inizializzati a '0', coerentemente con lo stato di reset (INIT) della FSM.
- I valori sovrascritti in RAM risultano corretti, verificando così lo scopo del modulo.
- o_done viene riportato a '0' quando i_start viene abbassato, un compito gestito dal modulo fsm_done.

Test Bench 2

Il secondo test è simile al primo, ma con una differenza: le prime parole in RAM sono tutte inizializzate a zero. Lo scopo è verificare che questi valori rimangano invariati (devono restare zero) e che la credibilità sia pari a 0 finché non si incontra un valore diverso da zero.

- Questo test è stato superato con successo, poiché std_value in FSM è determinato dall'ingresso fornito da reg_std_value. Dopo il reset, il valore iniziale di std_value è zero e rimane tale finché non viene aggiornato quando si incontra un valore diverso da zero.

Test Bench 3

In questo test sono state salvate due sequenze in RAM a indirizzi diversi. Le sequenze testate comprendono sia scenari simili al Test Bench 1, sia scenari del Test Bench 2 (con inizio della sequenza a zero). Tutte le combinazioni hanno prodotto risultati corretti.

- Al termine della prima sequenza, il modulo viene resettato con i_rst = '1' e poi riportato a zero. Una volta resettato, il comportamento del modulo è analogo ai test 1 e 2.

Test Bench 4

In questo test, durante l'esecuzione della prima sequenza, è stato inviato un segnale di reset in un punto qualsiasi dell'esecuzione. Cinquanta nanosecondi dopo, è stata avviata la seconda sequenza.

- Il test è stato superato, poiché tutti i segnali sono stati correttamente inizializzati, grazie alla gestione appropriata del segnale asincrono i_rst = '1', come descritto in precedenza.

Test Bench 5

Questo test riprende lo scenario del Test Bench 3, con l'unica differenza che è stato testato il caso limite in cui l'indirizzo della seconda sequenza sia consecutivo all'ultima parola della prima sequenza. Inoltre, non è stato eseguito alcun reset tra una sequenza e l'altra.

- Grazie al segnale init_m, tutti i componenti vengono correttamente inizializzati prima che i_start diventi '1'. Di conseguenza, la sequenza precedente non influisce in alcun modo sulla successiva.

Test Bench 6

In questo ultimo test, viene fornita una singola sequenza in input, come nel Test Bench 1. Tuttavia, in questo caso, la credibilità viene fatta decrementare fino a zero. Lo scopo è verificare che il valore di credibilità non scenda mai sotto zero.

- Il test è stato superato correttamente, grazie al comportamento del modulo dec, che impedisce ulteriori decrementi se dec_count = 0.

Conclusioni

La "Prova Finale (Progetto di Reti Logiche) 2023/24" ha portato allo sviluppo di un modulo VHDL in grado di interfacciarsi con una memoria RAM e modificarne i dati come previsto dalle specifiche. Attraverso una serie di test, eseguiti sia nella simulazione pre-sintesi che nella simulazione funzionale post-sintesi, è stato possibile verificare che il modulo gestisce correttamente il processo di aggiornamento della memoria.

Le simulazioni hanno mostrato come il modulo esegua con precisione le operazioni richieste, quali la gestione del reset e l'aggiornamento della RAM. Il comportamento desiderato è stato confermato in tutte le fasi di test, dimostrando il buon esito del progetto e la sua capacità di soddisfare i requisiti di progetto.